

Entregable Capitulo 02

Economia Computacional con Python

Tabla de contenidos

Capitulo 02. Instalacion, Google Colab y entorno de trabajo	2
1. Apertura	2
Pregunta motivadora	2
Caso aplicado	2
Por que importa	2
2. Resultados de aprendizaje	2
3. Conceptos clave	2
4. Intuicion economica	3
5. Formalizacion minima	3
6. Datos	3
7. Implementacion en Python	4
8. Resultados	4
9. Interpretacion economica	4
10. Errores frecuentes	4
11. Ejercicios	4
12. Mini-proyecto	5
13. Lecturas recomendadas	5
14. Resumen del capitulo	5
15. Checklist de aprendizaje	5
Anexo A. Ejercicios del estudiante	6
Ejercicios - Capitulo 2	6
Resultados de aprendizaje asociados	6
Ejercicios conceptuales	6
Ejercicios computacionales	6
Mini-proyecto	6
Criterios de entrega	6
Anexo B. Guia docente	7
Soluciones docentes - Capitulo 2	7
Criterios de revision	7
Guia orientativa por ejercicio	7
Ejercicio conceptual 1	7
Ejercicio conceptual 2	7
Ejercicio conceptual 3	7
Ejercicio conceptual 4	7
Ejercicio conceptual 5	7
Rubrica breve	7

Capitulo 02. Instalacion, Google Colab y entorno de trabajo

1. Apertura

Pregunta motivadora

Como preparamos un entorno de trabajo para que un analisis economico con Python pueda ejecutarse, revisarse y repetirse en otra computadora?

Caso aplicado

Un estudiante quiere analizar datos de empleo para su tesis. En su computadora el notebook corre bien, pero cuando lo envia al docente aparecen problemas: rutas absolutas, librerias no instaladas, archivos guardados en carpetas personales y resultados que no se pueden regenerar.

Este capitulo resuelve ese problema antes de trabajar con datos reales. El objetivo es construir una base tecnica simple, ordenada y portable.

Por que importa

En economia aplicada, la reproducibilidad no es un lujo tecnico. Es parte de la calidad del argumento. Si un resultado depende de una ruta local, una version no documentada de una libreria o una celda ejecutada fuera de orden, la evidencia queda debilitada.

Un buen entorno de trabajo permite que el analisis sea:

- verificable por otra persona;
- facil de ejecutar en clase;
- transferible entre computadora local y Google Colab;
- consistente entre capitulos;
- preparado para publicar notebooks, tablas y figuras.

2. Resultados de aprendizaje

Al finalizar este capitulo, el lector sera capaz de:

1. Diferenciar entre entorno local, entorno virtual, Conda y Google Colab.
2. Crear una estructura minima de proyecto reproducible.
3. Usar rutas relativas con `pathlib`.
4. Verificar la version de Python y librerias principales.
5. Explicar por que `requirements.txt` y `environment.yml` documentan el entorno.
6. Ejecutar un notebook de diagnostico y guardar un reporte en `outputs/reports`.

3. Conceptos clave

- Python.
- Jupyter Notebook.
- Google Colab.
- Entorno virtual.
- Conda.
- Dependencias.
- Kernel.
- Ruta relativa.
- Directorio raiz del proyecto.
- Reproducibilidad computacional.

4. Intuicion economica

La instalacion no es una pausa administrativa antes de hacer economia. Es el primer control de calidad del analisis empirico.

Cuando un economista estima una tabla, construye una variable o genera una figura, esta creando evidencia. Esa evidencia debe poder revisarse. Para lograrlo necesitamos tres acuerdos:

- donde estan los archivos;
- que version de herramientas se usa;
- que pasos regeneran los resultados.

Un entorno reproducible no elimina todos los errores, pero hace que los errores sean visibles y corregibles.

5. Formalizacion minima

El flujo de trabajo del libro sigue esta estructura:

```
ECONOLIBRO_CODEX/  
  chapters/  
  notebooks/  
  scripts/  
  data/  
    raw/  
    processed/  
  outputs/  
    tables/  
    figures/  
    reports/
```

La regla central es:

```
entrada original -> data/raw  
transformacion documentada -> scripts o notebooks  
salida derivada -> data/processed u outputs
```

Para rutas:

```
from pathlib import Path  
  
PROJECT_ROOT = Path.cwd()  
DATA_RAW = PROJECT_ROOT / "data" / "raw"  
OUTPUTS = PROJECT_ROOT / "outputs"
```

En notebooks ubicados dentro de subcarpetas, conviene detectar la raiz del proyecto buscando un archivo estable como `PROJECT_BRIEF.md`.

6. Datos

Este capitulo no usa datos economicos reales. Usa archivos de diagnostico del proyecto para comprobar que el entorno puede:

- identificar la raiz del repositorio;
- leer archivos de configuracion;
- crear carpetas de salida;
- escribir un reporte reproducible;

- respetar la regla de no modificar `data/raw`.

Los datos reales empezaran en capitulos posteriores, despues de documentar la fuente y su metodologia.

7. Implementacion en Python

El notebook asociado esta en:

```
notebooks/chapter_02/verificacion_entorno.ipynb
```

El notebook realiza cuatro tareas:

1. Detecta la raiz del proyecto.
2. Verifica version de Python y plataforma.
3. Revisa si existen carpetas y archivos clave.
4. Guarda un reporte de entorno en `outputs/reports`.

Este diagnostico ayuda a estudiantes y docentes a detectar problemas antes de trabajar con datos.

8. Resultados

El resultado esperado es un reporte de texto:

```
outputs/reports/chapter_02_entorno_python.txt
```

Tambien se genera una tabla de checklist:

```
outputs/tables/chapter_02_checklist_entorno.csv
```

Estas salidas no son evidencia economica. Son evidencia de reproducibilidad.

9. Interpretacion economica

Un entorno correctamente preparado no garantiza un buen analisis, pero reduce riesgos importantes:

- evita que los resultados dependan de una computadora especifica;
- facilita la revision docente;
- permite separar datos originales y productos derivados;
- hace mas claro el paso de notebook a publicacion.

La interpretacion economica vendra despues. En este capitulo evaluamos la infraestructura que la hara posible.

10. Errores frecuentes

- Instalar paquetes en un Python distinto al kernel usado por Jupyter.
- Usar rutas absolutas como `C:/Users/nombre/Desktop/...`
- Guardar outputs dentro de `data/raw`.
- Ejecutar celdas en desorden y no reiniciar el kernel.
- No registrar dependencias en `requirements.txt` o `environment.yml`.
- Mezclar notebooks de prueba con notebooks finales.
- Confundir Google Colab con reproducibilidad automatica.

11. Ejercicios

Los ejercicios estan en:

```
exercises/chapter_02/ejercicios.md
```

La guía docente esta en:

```
exercises/chapter_02/soluciones_docente.md
```

12. Mini-proyecto

El mini-proyecto consiste en preparar el entorno de trabajo personal para el libro y entregar:

- captura o texto con version de Python;
- confirmacion de que el notebook del capitulo corre de arriba hacia abajo;
- reporte generado en `outputs/reports`;
- checklist de carpetas y archivos clave;
- una nota breve explicando si trabajara localmente, en Colab o en ambos.

13. Lecturas recomendadas

Agregar solo referencias verificadas. Para esta unidad conviene consultar:

- documentacion oficial de Python sobre entornos virtuales;
- documentacion oficial de Jupyter;
- documentacion oficial de Google Colab;
- documentacion de pandas sobre instalacion cuando se use en capitulos posteriores.

14. Resumen del capitulo

Este capitulo preparo el terreno tecnico del libro. Aprendimos que un proyecto reproducible necesita estructura, rutas relativas, dependencias documentadas y notebooks que puedan ejecutarse desde cero.

La instalacion no debe verse como un obstaculo antes del analisis economico. Es el primer paso para que el analisis sea confiable, revisable y publicable.

15. Checklist de aprendizaje

- ☐ Puedo explicar que es un entorno de trabajo.
- ☐ Puedo diferenciar entorno local, Conda y Google Colab.
- ☐ Puedo usar rutas relativas con `pathlib`.
- ☐ Puedo verificar la version de Python desde un notebook.
- ☐ Puedo ejecutar el notebook del capitulo de arriba hacia abajo.
- ☐ Puedo ubicar outputs sin modificar `data/raw`.

Anexo A. Ejercicios del estudiante

Ejercicios - Capitulo 2

Resultados de aprendizaje asociados

- Preparar un entorno de trabajo reproducible.
- Verificar version de Python y kernel de Jupyter.
- Usar rutas relativas.
- Generar un reporte tecnico en `outputs/reports`.
- Explicar diferencias entre entorno local y Google Colab.

Ejercicios conceptuales

1. Explique por que una ruta absoluta dificulta la reproducibilidad.
2. Compare entorno local, entorno virtual, Conda y Google Colab. Indique una ventaja y una limitacion de cada opcion.
3. Explique por que `requirements.txt` y `environment.yml` no son archivos decorativos.
4. Describa que puede salir mal si Jupyter usa un kernel distinto al Python donde se instalaron los paquetes.
5. Explique por que `data/raw` no debe recibir archivos transformados.

Ejercicios computacionales

1. Ejecute `notebooks/chapter_02/verificacion_entorno.ipynb` de arriba hacia abajo.
2. Verifique que se cree `outputs/reports/chapter_02_entorno_python.txt`.
3. Verifique que se cree `outputs/tables/chapter_02_checklist_entorno.csv`.
4. Abra el reporte y confirme que muestre version de Python, plataforma y raiz del proyecto.
5. Reinicie el kernel y ejecute todo nuevamente para confirmar que las salidas se regeneran.

Mini-proyecto

Prepare una ficha de entorno personal con:

- sistema operativo;
- version de Python;
- forma de trabajo elegida: local, Conda, Google Colab o mixta;
- ruta relativa usada para outputs;
- problemas encontrados;
- solucion aplicada o duda pendiente.

Criterios de entrega

- El notebook debe ejecutarse completo.
- Las salidas deben generarse en `outputs`.
- No debe modificarse `data/raw`.
- La ficha debe distinguir instalacion, kernel y dependencias.
- La explicacion debe estar escrita para que otro estudiante pueda repetir el proceso.

Anexo B. Guia docente

Soluciones docentes - Capitulo 2

Criterios de revision

- El estudiante identifica correctamente la raiz del proyecto.
- El notebook se ejecuta de arriba hacia abajo.
- Las salidas se guardan en `outputs/reports` y `outputs/tables`.
- No se usan rutas absolutas.
- No se modifica `data/raw`.
- La respuesta distingue entorno local, entorno virtual, Conda y Colab.
- El estudiante entiende la diferencia entre instalar un paquete y usarlo desde el kernel correcto.

Guia orientativa por ejercicio

Ejercicio conceptual 1

Una ruta absoluta depende de la estructura de carpetas de una computadora particular. Una ruta relativa permite que el proyecto se mueva o comparta sin reescribir el codigo.

Ejercicio conceptual 2

Respuestas esperadas:

- Local: da control sobre archivos y dependencias, pero requiere instalacion.
- Entorno virtual: aisla dependencias, pero debe activarse correctamente.
- Conda: facilita dependencias cientificas, pero agrega una herramienta mas.
- Colab: reduce barreras de instalacion, pero depende de sesion remota y manejo cuidadoso de archivos.

Ejercicio conceptual 3

`requirements.txt` y `environment.yml` documentan el entorno. Permiten recrear dependencias y reducen ambigüedad entre computadoras.

Ejercicio conceptual 4

Puede ocurrir que el paquete este instalado en un Python, pero el notebook use otro kernel. El resultado tipico es `ModuleNotFoundError` aunque el estudiante crea que ya instalo la libreria.

Ejercicio conceptual 5

`data/raw` debe preservar los archivos originales. Las transformaciones deben quedar en `data/processed` para mantener trazabilidad y poder reconstruir el flujo.

Rubrica breve

Criterio	Excelente	A mejorar
Ejecucion	Notebook corre completo y regenera salidas	Requiere pasos manuales no documentados
Rutas	Usa rutas relativas	Usa rutas personales
Entorno	Distingue Python, kernel y dependencias	Mezcla conceptos
Reproducibilidad	Reporte y checklist verificables	Salidas ausentes o fuera de carpeta

Criterio	Excelente	A mejorar
Datos	Respeta <code>data/raw</code>	Escribe derivados en lugar incorrecto